



# 第四章 汇编语言程序设计

14:08

# 第四章 汇编语言程序设计



- 4.1 汇编语言程序设计概述
- 4.2 汇编语言基本语法（重点）
- 4.3 汇编语言程序设计（重难点）

14:08

# 第四章 汇编语言程序设计



## 4.1 汇编语言程序设计概述

## 4.2 汇编语言基本语法（重点）

## 4.3 汇编语言程序设计（重难点）

14:08

## 4.1 汇编语言程序设计概述



一、程序设计语言

二、汇编语言源程序

三、汇编语言程序开发过程



## 4.1 汇编语言程序设计概述



一、程序设计语言

二、汇编语言源程序

三、汇编语言程序开发过程

# 一、程序设计语言



## 1、机器语言

用二进制表示，能够被机器直接识别

## 2、汇编语言

采用助记符表示机器指令

## 3、高级语言

接近自然语言

## 4.1 汇编语言程序设计概述



一、程序设计语言

二、汇编语言源程序

三、汇编语言程序开发过程

## 二、汇编语言源程序



- 汇编语言源程序是程序员根据具体问题的算法，用汇编语言的语句编写的程序文本，通常以`.asm`作为扩展文件名。
- 汇编语言程序设计中由于开发环境支持不同，可以有完整段程序和简化段程序。

## 二、汇编语言源程序



完整段程序：

```
STACK      SEGMENT
```

```
    DW 256 DUP(?)
```

```
STACK      ENDS
```

```
DATA       SEGMENT
```

```
    AA DB 12,25,36
```

```
DATA       ENDS
```

```
CODE       SEGMENT
```

```
    ASSUME CS:CODE,DS:DATA,SS:STACK
```

```
START:     .....
```

```
CODE       ENDS
```

```
END        START
```

## 二、汇编语言源程序



简化源程序：

**.MODEL**      模式名

**.DATA**

.....

**.STACK**

**.CODE**

**.STARTUP**

.....

**.EXIT**

.....

**END**

## 4.1 汇编语言程序设计概述



一、程序设计语言

二、汇编语言源程序

三、汇编语言程序开发过程

# 三、汇编语言程序开发过程



基本步骤:

①编辑: **XXX.ASM**

②汇编: **XXX.OBJ**

③连接: **XXX.EXE**

④调试运行



# 三、汇编语言程序开发过程



## ①编辑：保存为A1.ASM

```
01 DATAS SEGMENT
02     BUF DB 'HELLO$'
03 DATAS ENDS
04
05 CODES SEGMENT
06     ASSUME CS:CODES,DS:DATAS
07 START:
08     MOV AX,DATAS
09     MOV DS,AX
10     MOV DX,OFFSET BUF
11     MOV AH,9
12     INT 21H
13     MOV AH,4CH
14     INT 21H
15 CODES ENDS
16     END START
```

```
DATAS SEGMENT
    BUF DB 'HELLO$'
DATAS ENDS
```

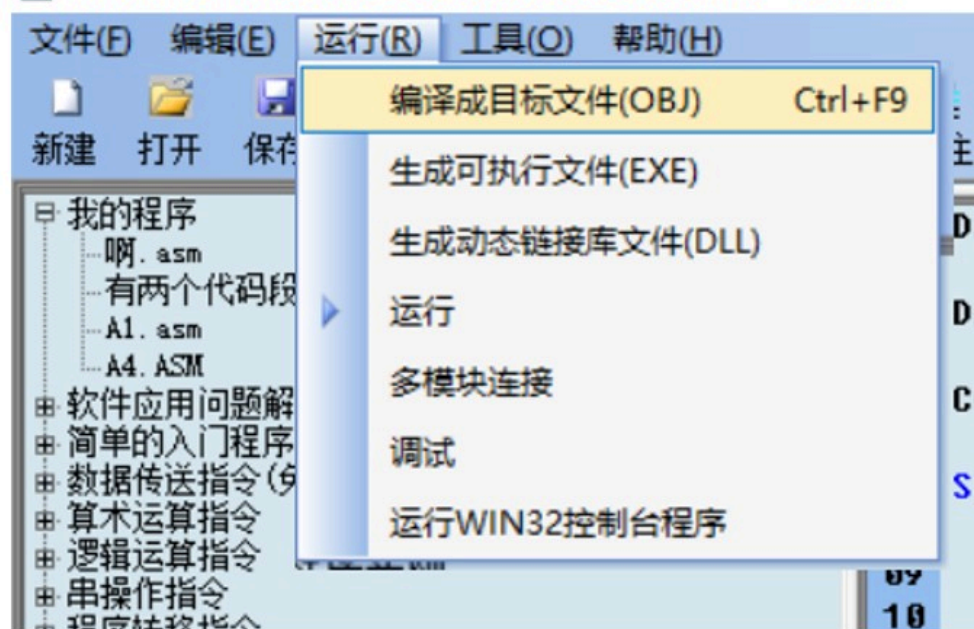
```
CODES SEGMENT
    ASSUME CS:CODES,DS:DATAS
START:
    MOV AX,DATAS
    MOV DS,AX
    MOV     DX,OFFSET BUF
    MOV AH,9
    INT 21H
    MOV AH,4CH
    INT 21H
CODES ENDS
    END START
```

# 三、汇编语言程序开发过程



## ②汇编：生成A1.OBJ

Masm for Windows 集成实验环境共享版 2015 - A1.asm



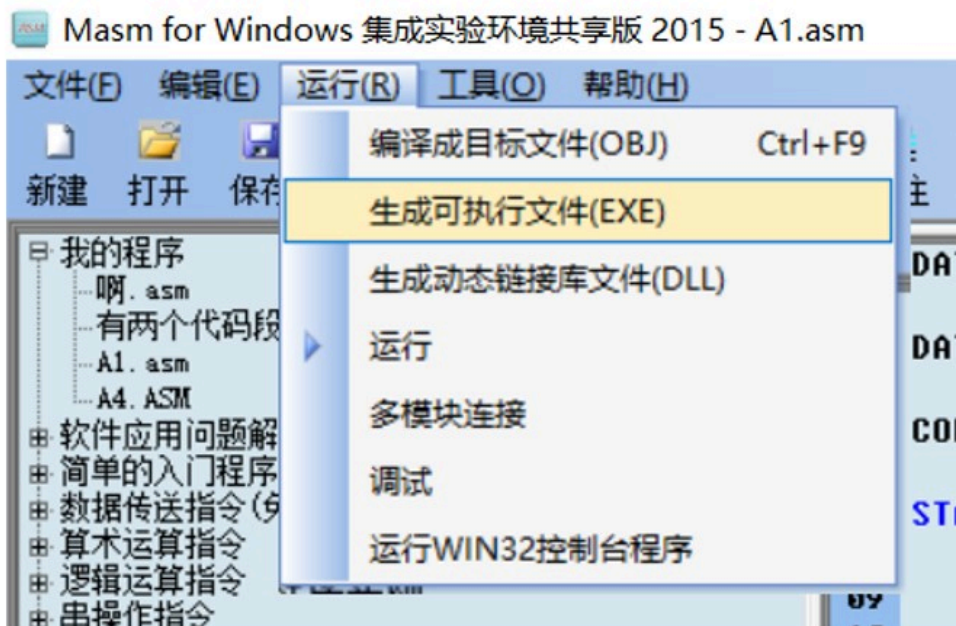
不成功会有  
错误提示

【兼容WinXP模式】编译源程序 C:\JMSOFT\Masm\A1.asm  
编译生成OBJ文件C:\JMSOFT\Masm\A1.obj 成功!

# 三、汇编语言程序开发过程



## ③连接：生成A1.EXE



【兼容WinXP模式】编译源程序 C:\JMSOFT\Masm\A1.asm  
生成EXE文件C:\JMSOFT\Masm\A1.exe 成功!

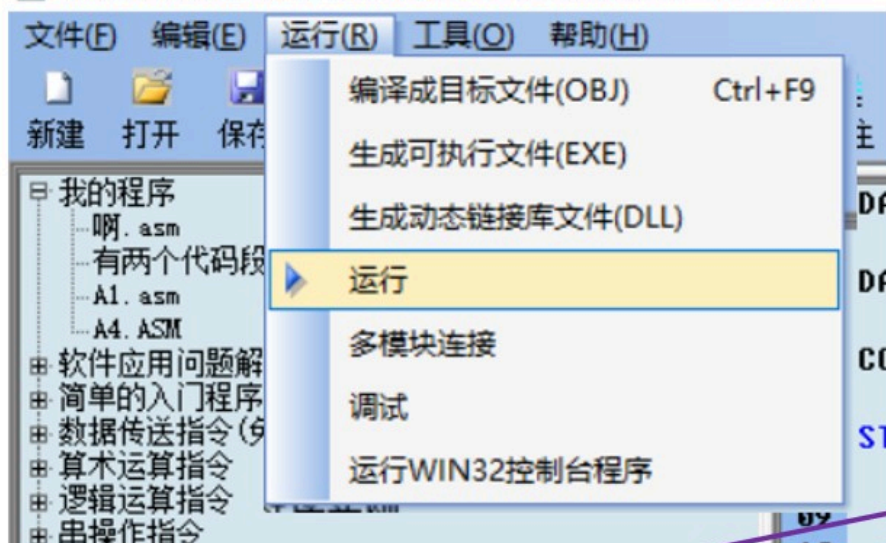


# 三、汇编语言程序开发过程

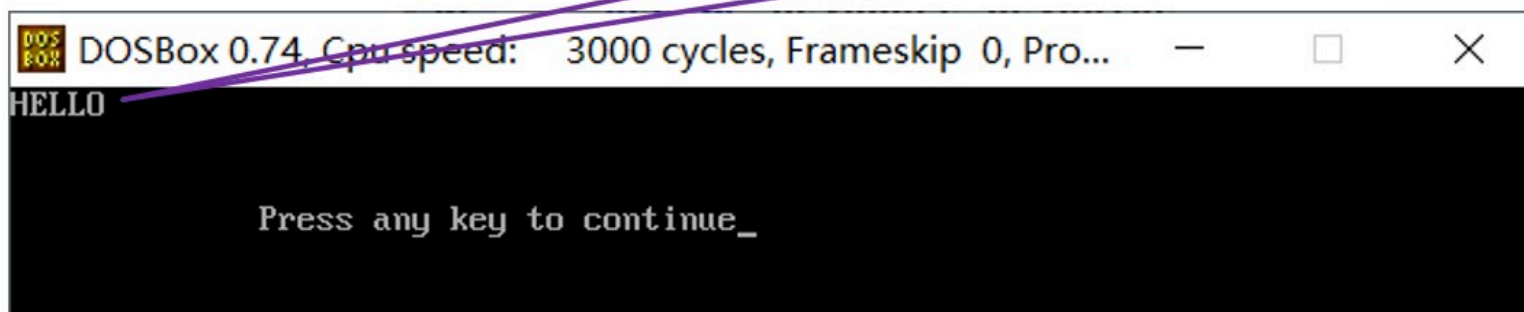


## ④调试运行

Masm for Windows 集成实验环境共享版 2015 - A1.asm



输出  
“HELLO”



# 第四章 汇编语言程序设计



## 4.1 汇编语言程序设计概述

## 4.2 汇编语言基本语法（重点）

## 4.3 汇编语言程序设计（重难点）

14:08

## 4.2 汇编语言基本语法



- 一、汇编语言的语句类型
- 二、常量、标识符和表达式
- 三、汇编语言程序伪指令（重点）
- 四、**DOS**系统功能调用（重点）

## 4.2 汇编语言基本语法



一、汇编语言的语句类型

二、常量、标识符和表达式

三、汇编语言程序伪指令（重点）

四、DOS系统功能调用（重点）

# 一、汇编语言的语句类型



**1、指令性语句：**由指令构成，与机器指令一一对应。

**[标号:] 操作码      操作数      [;注释]**

如: **START:    MOV AX,DATA    ;初始化数据段**

**2、指示性语句：**由命令（伪指令）构成，是程序员发给汇编程序的命令，没有相应的机器指令。

**[名字]    伪指令      [操作数]      [;注释]**

如: **DATA      SEGMENT**



## 4.2 汇编语言基本语法



一、汇编语言的语句类型

二、常量、标识符和表达式

三、汇编语言程序伪指令（重点）

四、DOS系统功能调用（重点）

## 二、常量、标识符和表达式



### 1、常量

指令中出现的固定值，在程序运行期间不会变化。分为数字常量、字符常量和符号常量三种。(指令中的立即数,**MEM**直接地址)

## 二、常量、标识符和表达式



(1) 数字常量：直接用数字表示的常量。

二进制：10000100B、11110001020100B

十进制：12356D

十六进制：12ABH、0F56AH（字母打头必须前面补0，否则将出现汇编语法错误）

例如：MOV AX, 100D

MOV BL, 0FAH

## 二、常量、标识符和表达式



**(2) 字符串常量：**包含在单引号内的一个或两个ASCII字符构成，它所代表的数值就是该字符的ASCII码。

例如字符'A'等价于41H，字符'AB'等价于4142H。

例如：     **MOV    AL,'A'**

**MOV    AX,'AB'**

**(3) 符号常量：**用标识符（常量名）表示的常量，它具有一个设定的数值从而被引用。

## 二、常量、标识符和表达式



### 2、标识符

有特定意义的字符序列，标识符可用作符号常量、名字、变量和标号等。

## 二、常量、标识符和表达式



标识符命名规则：

- **0≤31个ASCII码字符**
- **A-Z、a-z、0-9、?、@、\$及下划线构成，除数字以外，所有规定的字符都可作为标识符的第一个字符。**
- **? 不能单独作为标识符。**
- **无独立的保留字及运算符。**



## 二、常量、标识符和表达式



合法标识符:

**STA\_124\$**

**MOV\_?**

**@103**

非法标识符:

**STA+124\$**

**MOV**

**?**

## 二、常量、标识符和表达式



### 3、表达式

由操作数（常量、变量、标号）和运算符构成。

在汇编时完成相应的运算(数字常数)，（**OBJ**）目标程序中不存在表达式，应用程序的**DEBUG**调试中不可能看到任何表达式。

## 二、常量、标识符和表达式



|     |   |   |                                      |
|-----|---|---|--------------------------------------|
| 优先级 | 高 | 1 | 括号中的项，即 (...) 和 [...]                |
|     |   | 2 | LENGTH, SIZE, WIDTH, MASK            |
|     |   | 3 | <b>PTR, OFFSET, SEG</b> , TYPE, THIS |
|     |   | 4 | *, /, MOD                            |
|     |   | 5 | +, -                                 |
|     |   | 6 | EQ, NE, LT, LE, GT, GE               |
|     |   | 7 | NOT                                  |
|     |   | 8 | AND                                  |
|     | 低 | 9 | OR, XOR                              |

## 二、常量、标识符和表达式



例如: **AND**      **AL, 86H AND 0FH**

**AND**是指令;

**AND**是逻辑运算符, **86H AND 0FH=06H**

汇编后的指令是: **AND**      **AL, 06H**

注意:

- ✓ 逻辑运算符只能对常数（或相当于常数）进行运算。
- ✓ 与逻辑运算指令不同, **CPU**不执行任何操作, 汇编时运算, 在目标程序中只是一个常数。

## 4.2 汇编语言基本语法



一、汇编语言的语句类型

二、常量、标识符和表达式

三、汇编语言程序伪指令（重点）

四、DOS系统功能调用（重点）

### 三、汇编语言程序伪指令



- 程序结束伪指令
- 段定义伪指令
- 过程定义的伪指令
- 数据定义伪指令
- 符号定义伪指令
- 变量



# 三、汇编语言程序伪指令



## 1、程序结束伪指令 **END**

**格式:** **END**        标号

**说明:** 标号为程序中第一条指令性语句标号

### 三、汇编语言程序伪指令



完整段程序：

**STACK        SEGMENT**

**DW 256    DUP(?)**

**STACK        ENDS**

**DATA         SEGMENT**

**AA DB 12,25,36**

**DATA         ENDS**

**CODE         SEGMENT**

**ASSUME CS:CODE,DS:DATA,SS:STACK**

**START:       .....**

**CODE         ENDS**

**END           START**

# 三、汇编语言程序伪指令



## 2、段定义伪指令

8086分段访问存储器。

### (1) 段定义语句

格式:

段名 **SEGMENT** [定位方式][组合方式][‘类别名’]

...

段名 **ENDS**

### 三、汇编语言程序伪指令



例如：

```
DATA    SEGMENT
```

```
...
```

```
DATA    ENDS
```

```
CODE    SEGMENT
```

```
...
```

```
CODE    ENDS
```

定义了二个段，段名分别为**DATA**、**CODE**。

**说明：**定义数据段、附加数据段和堆栈段时，处于**SEGMENT/ENDS**伪指令中间的语句，只能包括伪指令语句，不能包括指令语句。

# 三、汇编语言程序伪指令



## (2) 段寄存器说明伪指令

**格式:** **ASSUME** 段寄存器: 段名1, 段寄存器: 段名2.....

**说明:** 在代码段, 告诉汇编程序**CS**、**DS**、**ES**、**SS**应具有的符号段基址, 但是段寄存器 (**CS**除外) 还必须用传送指令赋值。一般紧跟在**SEGMENT**语句之后。



### 三、汇编语言程序伪指令



完整段程序：

**STACK        SEGMENT**

**DW 256    DUP(?)**

**STACK        ENDS**

**DATA         SEGMENT**

**AA DB 12,25,36**

**DATA         ENDS**

**CODE         SEGMENT**

**ASSUME CS:CODE,DS:DATA,SS:STACK**

**START:        .....**

**CODE         ENDS**

**END           START**



## 三、汇编语言程序伪指令



### 3、过程定义伪指令

**格式：**        过程名 **PROC**    **NEAR[FAR]**

.....

过程名 **ENDP**

**说明：** **NEAR** 近过程（主、子同段）

**FAR** 远过程（主、子在两个不同的代码段）

# 三、汇编语言程序伪指令



## 4、数据定义伪指令

数据定义语句用来为数据分配存储单元，数据段、附加段和堆栈段都是存放数据的，其中所用的语句主要是数据定义语句。

**格式：**

**[变量名] 命令 参数1，参数2，... [； 注释]**

**说明：**

➤ **变量名：** 由程序员在编程时按标识符规定取的。

### 三、汇编语言程序伪指令



➤ 命令：表示符号及功能如下：

|           |    |                                    |
|-----------|----|------------------------------------|
| <b>DB</b> | 字节 | ( <b>8</b> 位) 一个字节存储单元             |
| <b>DW</b> | 字  | ( <b>16</b> 位) 二个连续字节存储单元          |
| <b>DD</b> | 双字 | ( <b>32</b> 位) 四个连续字节存储单元          |
| <b>DQ</b> | 双字 | ( <b>64</b> 位) <b>8</b> 个连续字节存储单元  |
| <b>DT</b> | 双字 | ( <b>80</b> 位) <b>10</b> 个连续字节存储单元 |

### 三、汇编语言程序伪指令



➤参数：相应内存单元中的数据，可以是数字常量、字符常量或符号常量，当它是保留时就以问号（？）表示。参数可以有多个，相互间用逗号（，）隔开，若连续多个数据是重复的，可用复制符**DUP**以简化书写，**DUP**的用法为：

复制次数 **DUP**（数据）

例如：        **ARRAY DB 10 DUP(12H)**

### 三、汇编语言程序伪指令



例1：段名为**DATA**的段由以下语句组成

**DATA SEGMENT**

**DATA1 DB 20H**

**ARRAY DB 12H,12,'A'**

**SUM DB ?**

**DATA ENDS**

设本段的段基址为**2000H**，则相应的内存分配为：



### 三、汇编语言程序伪指令



**DATA1 DB 20H**

段基址=**2000H**

**ARRAY DB 12H,12,'A'**

**SUM DB ?**

偏移地址

0000H

20H

0001H

12H

0002H

0CH

0003H

41H

0004H

?

问题：如何把**ARRAY**的第一个元素送给**AL**？

**MOV AX,2000H**

**MOV DS,AX**

**MOV AL,[OFFSET ARRAY]**

**MOV AL,ARRAY**



### 三、汇编语言程序伪指令



**例2: DATA2 DB 2 DUP (12H, 34H, 56H)**

其内存分配为

| 偏移地址  |     |
|-------|-----|
| 0000H | 12H |
| 0001H | 34H |
| 0002H | 56H |
| 0003H | 12H |
| 0004H | 34H |
| 0005H | 56H |

### 三、汇编语言程序伪指令



例3: **DATA3 DB 'ABCD'**

其中，参数部分 '**ABCD**' 是 '**A**', '**B**', '**C**', '**D**' 的简写。

其内存分配为

| 偏移地址  |     |
|-------|-----|
| 0000H | 41H |
| 0001H | 42H |
| 0002H | 43H |
| 0003H | 44H |

### 三、汇编语言程序伪指令



例4: **DATA4 DW 1234H, 9AH, ?**  
其内存分配为

| 偏移地址  |     |
|-------|-----|
| 0000H | 34H |
| 0001H | 12H |
| 0002H | 9AH |
| 0003H | 00H |
| 0004H | ?   |
| 0005H | ?   |

### 三、汇编语言程序伪指令



例5: **DATA5 DW 'AB', 'CD'** ;不能写为'**ABCD**'  
其内存分配为

| 偏移地址  |     |
|-------|-----|
| 0000H | 42H |
| 0001H | 41H |
| 0002H | 44H |
| 0003H | 43H |

**注意:** 除用**DB**定义的字符串常量外, 单引号中**ASCII**字符的个数不得超过**2**个, 若只有一个, 如 **DW 'C'**, 就相当于 **DW 0043H**。

## 课堂练习10



画出下列语句中的数据在存储器中的存储情况

**ARRAYB DB 63,63H,'ABCD',3 DUP(?),2 DUP(1,3)**

**ARRAYW DW 1234H,5,'AB','CD',?,2 DUP(1,3)**

# 三、汇编语言程序伪指令



## 5、符号定义伪指令

### (1) 应用命令**EQU**和**PURGE**

**格式：** 符号常量 **EQU** 表达式

**说明：** 如需对已赋值的名字赋以新值，则先用**PURGE**语句撤销原赋值，其格式为：

**PURGE** 名字

**PURGE**可同时撤销几个已赋值。



### 三、汇编语言程序伪指令



例如：

|              |                  |
|--------------|------------------|
| <b>COUNT</b> | <b>EQU 20</b>    |
| <b>ADD</b>   | <b>AX, COUNT</b> |
| <b>PURGE</b> | <b>COUNT</b>     |
| <b>COUNT</b> | <b>EQU 30</b>    |
| <b>ADD</b>   | <b>AX, COUNT</b> |

## 三、汇编语言程序伪指令



### (2) 应用命令 “=”

**格式：** 符号常量=表达式

**说明：** 其功能与**EQU**类似，唯一的区别是命令“=”可随时对名字赋新值，而不必使用**PURGE**命令。

**例：**

**COUNT=20**

**ADD AX, COUNT**

**COUNT=30**

**ADD AX, COUNT**

# 三、汇编语言程序伪指令



## 6、变量

### (1) "\$"的含义

**DATA        SEGMENT**

**ARRAY DB 10H, 5AH, 0C7H, 98H, 'ABCD'**

**COUNT EQU \$-ARRAY**

**MAX        DB 12H**

**DATA        ENDS**

**说明：**\$表示该行的偏移地址，此处**COUNT**表示**ARRAY**有多少个元素。

# 三、汇编语言程序伪指令



## (2) 变量的五种属性

### ➤ 段基址

**格式:** **SEG**      符号名

**例:**      **MOV AX, SEG AA**

**说明:** **SEG AA** 为立即寻址, 是**AA**的段地址

### ➤ 偏移地址

**格式:** **OFFSET**   符号名

**例:**      **MOV SI, OFFSET BB**

**说明:** **OFFSET BB** 为立即寻址, 是**BB**的偏移地址

### 三、汇编语言程序伪指令



#### ➤ 求符号名的类型值

格式: **TYPE**      符号名

#### ➤ 长度属性

格式: **LENGTH**    符号名

#### ➤ 规模属性

格式: **SIZE**      符号名

## 三、汇编语言程序伪指令



### (3) 类型指定运算符PTR

**格式：**类型 **PTR** 表达式

**说明：**类型可以是**BYTE**、**WORD**、**DWORD**、**NEAR**、**FAR**

**作用：**按**PTR**前面指定的类型去寻址

**例：** **INC** **BYTE PTR** **[2000H]**



## 4.2 汇编语言基本语法



- 一、汇编语言的语句类型
- 二、常量、标识符和表达式
- 三、汇编语言程序伪指令（重点）
- 四、**DOS系统功能调用（重点）**

## 四、DOS系统功能调用



### 什么叫DOS系统功能调用？

为提高汇编程序的编程效率，减少重复开发过程，**MS-DOS**操作系统内置了几十个子程序，这些子程序能够完成大量底层功能，用户程序可以通过软中断的方式使用这些子程序，软中断号为**21H**。指令为 **INT 21H**。

## 四、DOS系统功能调用



**DOS**系统功能子程序使用的基本要求如下：

- 1) 传送入口参数到指定寄存器中；
- 2) 调用子功能的功能编号放在**AH**寄存器中；
- 3) 执行**INT 21H**指令。

## 四、DOS系统功能调用



### 1、键盘单字符输入（1号）

**MOV AH, 1**

**INT 21H**

**功能：**等待从键盘输入一个字符。

**返回：**（**AL**）=**ASCII**码，并回显在显示器上。

## 四、DOS系统功能调用



### 2、输出单字符（2号）

**MOV DL, 'A'**

**MOV AH, 2**

**INT 21H**

**功能：**将**DL**中字符从屏幕上输出。

**返回：**无返回

## 四、DOS系统功能调用



### 3、输出字符串（09号）

**MOV DX, OFFSET BUF**

**MOV AH, 09**

**INT 21H**

**功能：**BUF中以'\$'为结束标志的字符串显示在屏幕上。当无结束标志时会出现乱码。



## 四、DOS系统功能调用



### 4、键盘输入字符串（0AH号）

```
MOV    DX, OFFSET BUF
```

```
MOV    AH, 0AH
```

```
INT     21H
```

**功能：**等待从键盘输入一串字符，回车结束。字符串存入键盘缓冲区。

键盘缓冲区定义：

```
BUF    DB    81    ;缓冲区的大小
```

```
DB     ?          ;实际输入的字符个数
```

```
DB     80    DUP (?) ;字符串存放区
```

## 四、DOS系统功能调用



### 5、返回操作系统（4CH）

**MOV AH, 4CH**

**INT 21H**

**功能：** 将控制权交给操作系统。

# 第四章 汇编语言程序设计



## 4.1 汇编语言程序设计概述

## 4.2 汇编语言基本语法（重点）

## 4.3 汇编语言程序设计（重难点）

14:08

## 4.3 汇编语言程序设计



程序设计的一般步骤：

- 1) 分析问题
- 2) 确定算法
- 3) 分配存贮区
- 4) 绘制流程图
- 5) 根据流程图编程
- 6) 调试程序。

## 4.3 汇编语言程序设计



- 顺序结构设计
- 分支结构设计
- 循环结构设计
- 子程序设计

## 4.3 汇编语言程序设计



例6：试编制一程序，求出下列公式中的 $Z$ 值，并存放在**RESULT**单元中。其中， $X$ 、 $Y$ 的值分别存放在**VARX**、**VARY**单元中。

$$Z = \frac{(X+Y) \times 8 - X}{2}$$



## 4.3 汇编语言程序设计



```
DATA          SEGMENT
    VARX      DW      6
    VARY      DW      7
    RESULT    DW      ?
DATA          ENDS
COSEG         SEGMENT
    ASSUME     CS:COSEG,DS:DATA
START:MOV     AX, DATA
    MOV       DS, AX
    MOV       DX, VARX
    ADD       DX, VARY
    MOV       CL, 3
    SAL       DX, CL
    SUB       DX, VARX
    SAR       DX, 1
    MOV       RESULT, DX
    MOV       AH, 4CH
    INT       21H
COSEG         ENDS
END           START
```

## 4.3 汇编语言程序设计



**例7：**内存中自**TABLESQ**开始的**10**个单元连续存放着自然数**0**到**9**的平方值，任给一数**x**( $0 \leq x \leq 9$ ) 在**XX**单元中，查表求出**x**的平方值，将结果存入**YY**单元中。

## 4.3 汇编语言程序设计



```
DATA          SEGMENT
    TABLESQ  DB    0,1,4,9,16,25,36,49,64,81
    XX        DB    ?
    YY        DB    ?
DATA          ENDS
CODE          SEGMENT
    ASSUME    CS:CODE,DS:DATA
START:        MOV    AX,DATA
              MOV    DS,AX
              MOV    AH,1
              INT     21H
              AND     AL,0FH
              MOV     XX,AL
              MOV     BX,OFFSET TABLESQ
              XLAT
              MOV     YY,AL
              MOV     AH,4CH
              INT     21H
CODE          ENDS
              END     START
```

## 4.3 汇编语言程序设计



**例8：**从键盘输入字符串并显示在显示器上。

**思路：**DOS的**9**号功能调用和**10**号功能调用

## 4.3 汇编语言程序设计



```
DATA        SEGMENT
    KEYBUF   DB    12,?,11 DUP('$')
DATA ENDS
CODE        SEGMENT
    ASSUME   CS:CODE,DS:DATA
START:      MOV    AX,DATA
            MOV    DS,AX
            LEA    DX,KEYBUF
            MOV    AH,0AH
            INT    21H
            LEA    DX,KEYBUF+2
            MOV    AH,09H
            INT    21H
            MOV    AH,4CH
            INT    21H
CODE        ENDS
            END    START
```



## 4.3 汇编语言程序设计



**例9：**已知有**N**个数据存放在以**BUF**为首地址的字节存储区中，编程统计其中负数的个数。

**思路：**循环次数已知，采用计数型循环，在程序中用到三个寄存器：

**AX：**用来累加**BUF**中负数的个数，初值为**0**

**BX：**指示**BUF**的地址，初值为**BUF**的首地址

**CX：**控制循环的次数，初值为**N**

循环条件：当**CX≠0**时，执行循环体



## 4.3 汇编语言程序设计



```
DATA          SEGMENT
    BUF        DB      -2,5,-3,6,10,0,-20,-9
    N=$-BUF
    RESULT      DW      ?
DATA          ENDS
CODE          SEGMENT
    ASSUME      CS:CODE,DS:DATA
START:        MOV     AX,DATA
              MOV     DS,AX
              MOV     AX,0
              LEA     BX,BUF
              MOV     CX,N
CYCLE:        CMP     BYTE PTR [BX],0
              JGE     NEXT
              INC     AX
NEXT:         INC     BX
              LOOP    CYCLE
              MOV     RESULT,AX
              MOV     AH,4CH
              INT     21H
CODE          ENDS
END           START
```

## 4.3 汇编语言程序设计



### 子程序的调用

**格式:** **CALL**      过程名（子程序地址）

**功能:**

1) 下条指令的地址压入堆栈。

主子同段（段内调用）：

只将**IP**的值压入堆栈

段间调用：

先将**CS**的值压入堆栈，再将**IP**的值压入堆栈。

2) 转入子程序运行（子程序的地址送入**CS: IP**）

## 4.3 汇编语言程序设计



### 子程序的返回

**格式: RET**

**说明:** 子程序的最后一条指令，用于返回**CALL**指令的下条指令继续执行。无论对那一种调用方式其返回指令都相同。

**功能:**

- 1) 段内调用只将当前**[SP]**弹进**IP**，从而使程序正常返回。
- 2) 对于段间调用则先弹**IP**，再弹**CS**。

## 4.3 汇编语言程序设计



### 子程序的设计与应用应注意的问题

#### 1) 现场的保护

```
PUSH    AX  
PUSH    BX  
PUSH    SI  
  
.....  
  
.....  
POP     SI  
POP     BX  
POP     AX  
RET
```

## 4.3 汇编语言程序设计



### 2) 参数的传递

主程序调用子程序必须传递入口参数，  
子程序返回必须传递出口参数。

常用的方法有：

- 寄存器：适用于参数少的情况。
- 约定单元：适用于参数多的情况。要事先建立参数数据缓冲区。
- 堆栈：适用于参数较多，且子程序嵌套，递归调用的情况



## 4.3 汇编语言程序设计



**例10：** 2组8位无符号数，每组中有N个无符号数，分别找出每一组中的最大数，并将最大数存放在每组数的开始地址。

**思路：** 编写一个子程序求N个数中的最大数，再两次调用该子程序求这两组数中的最大数。

DATA SEGMENT

BUF1 DB ?,12H,34H,26H,60H,22H

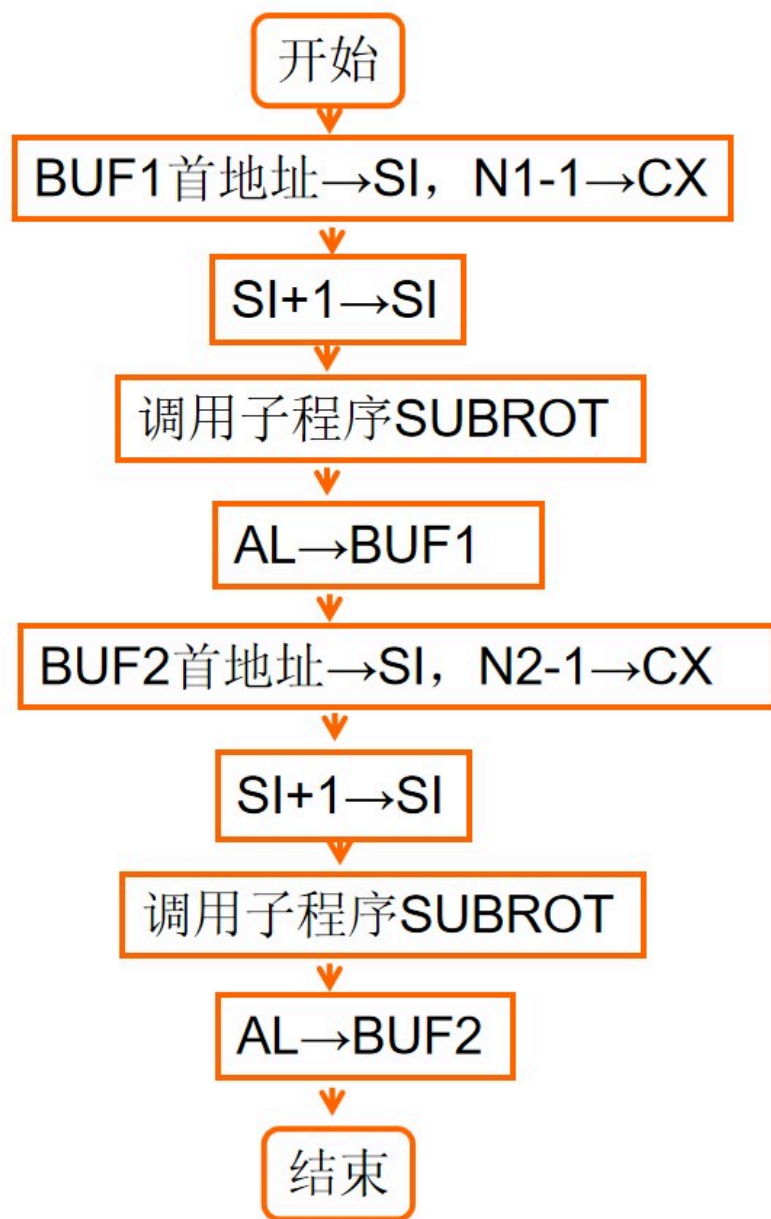
N1=\$-BUF1

BUF2 DB ?,16H,68H,47H,55H

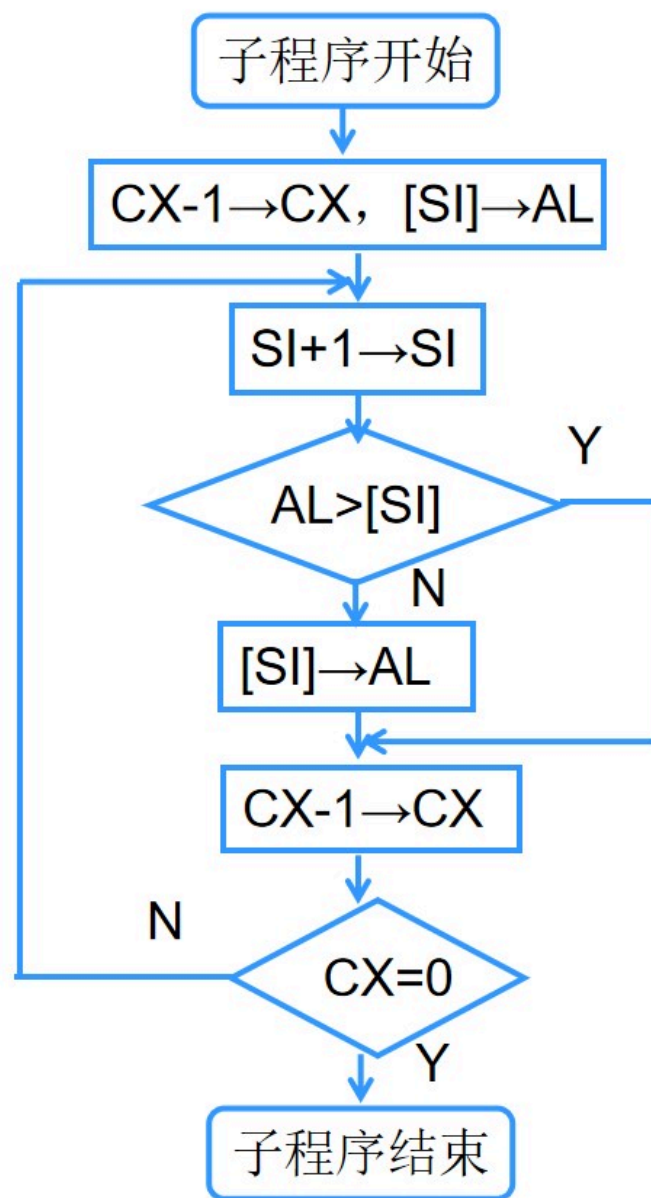
N2=\$-BUF2

DATA ENDS





例10流程图



例10子程序流程图

```

DATA    SEGMENT
    BUF1  DB ?,12H,34H,26H,60H,22H
    N1=$-BUF1
    BUF2  DB ?,16H,68H,47H,55H
    N2=$-BUF2
DATA    ENDS
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA
START:  MOV    AX,DATA
        MOV    DS,AX
        LEA    SI,BUF1
        INC    SI
        MOV    CX,N1- 1
        CALL   SUBROT
        MOV    BUF1,AL
        LEA    SI,BUF2
        INC    SI

```

```

        MOV    CX,N2
        DEC    CX
        CALL   SUBROT
        MOV    BUF2,AL
        MOV    AH,4CH
        INT    21H

SUBROT   PROC
        DEC    CX
        MOV    AL,[SI]
LOOP1:  INC    SI
        CMP    AL,[SI]
        JA     NEXT
        MOV    AL,[SI]
NEXT:   LOOP   LOOP1
        RET

SUBROT   ENDP
CODE    ENDS
        END    START

```

## 4.3 汇编语言程序设计



**例11：** 编写含有两个代码段的程序，其中一个代码段**MY**包含在显示器上输出“**HELLO**”的子程序**SUB1**，在另一个代码段**CODE**里调用**SUB1**在显示器上输出“**HELLO**”。

## 4.3 汇编语言程序设计



```
DATA SEGMENT
    BUF DB 'HELLO$'
DATA ENDS
MY SEGMENT
    ASSUME CS:MY
SUB1 PROC FAR
    MOV DX,OFFSET BUF
    MOV AH,9
    INT 21H
    RET
SUB1 ENDP
MY ENDS
```

```
CODE SEGMENT
    ASSUME CS:CODE,DS:DATA
START:
    MOV AX,DATA
    MOV DS,AX
    CALL SUB1
    MOV AH,4CH
    INT 21H
CODE ENDS
END START
```